

# Lecture 22

05/11/24 10:10 AM

$\Delta \leftarrow$  threshold (initialise to  $C$ )

$G(\Delta)$  = keep the edges of  $G$  with capacity  $\geq \Delta$   
delete the rest

- Find s-t paths in  $G(\Delta)$ .
  - Augment the flow by the bottleneck.
  - Construct a res. graph. and ensure edges have capacity  $\geq \Delta$
  - Repeat until there is no s-t path.
- }  $L$  iterations  
 $\Rightarrow F \geq L \cdot \Delta$

Repeat for  $\frac{\Delta}{2}$ .

$$\text{Let } C = \min \left\{ \sum_{s \rightarrow u \in E} c(s \rightarrow u), \sum_{w \rightarrow t \in E} c(w \rightarrow t) \right\}$$

Original algo:  $O(|C|(m+n))$

New algo:  $\log |C| \times \text{max \# of iterations} \times O(m+n)$   
no. of  $\frac{\Delta}{2^i}$  phases  $\underbrace{\text{in each } \frac{\Delta}{2^i} \text{ phase}}_{\geq 2m}$

$$= O(\log |C| \times m \times O(m+n))$$

$\Rightarrow$  New algo performs better if  $\log |C| \times m < |C|$   
Choose algo accordingly.

Init:  $\Delta \leftarrow$  largest power of 2 s.t. it is smaller than  $|C|$   
 $\frac{|C|}{2} \leq \Delta \leq |C|$   
 $\Delta = \max \{2^k | 2^k < |C|\}$  } preprocessing

Construct the graph  $G(\Delta)$ .

- Find an s-t path in  $G(\Delta)$  if it exists.

**YES**

Update the flow.  
 $F \leftarrow F + \text{bottleneck}$

Compute the residual graph.  
Remove edges of capacity  $< \Delta$ .  
 $O(m+n)$  bookkeeping  
to maintain flows and capacities in the original graph

Go to step 1.

**NO**

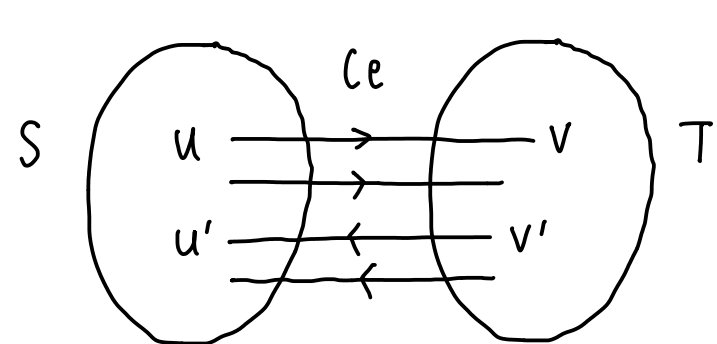
Update  $\Delta \leftarrow \Delta / 2$ .  
Update the residual graph by including edges of  $\geq \Delta$ .

If  $\Delta = 1$  and there are no s-t paths, return  $F$ .

Need to bound the no. of iterations in each  $\Delta$  phase.

Lemma: No. of augmentations in each  $\Delta$ -phase  $\leq 2m$

Claim: If  $F$  be the flow at the end of a  $\Delta$ -phase, then capacity of the cut obtained at the end of the phase is at most  $F + m \cdot \Delta$ .



\* CASE 1:  $u \rightarrow v \in E(G_{\text{orig}})$

Then  $C_e < f_e + \Delta$

If not, then  $C_e \geq f_e + \Delta$

$\Rightarrow$  There would be a residual capacity  $\geq \Delta$

Algo would not have terminated since this would lead to an s-t path or  $v$  would be part of  $s$  itself.

\* CASE 2:  $v' \rightarrow u' \in E(G_{\text{orig}})$

Then  $f_e < \Delta$

$$\text{Flow} = \sum_{\substack{\text{fwd edges} \\ S \rightarrow T}} f_e - \sum_{\substack{e': \\ \text{back edges}}} f_{e'} > \sum_{\substack{e \\ \text{fwd}}} (C_e - \Delta) - \sum_{\substack{e' \\ \text{back}}} \Delta > \sum_{\substack{e \\ \text{fwd}}} C_e - \underbrace{\sum_{e, e'} \Delta}_{\leq m \cdot \Delta}$$

$$\therefore \text{Flow} > \sum_e C_e - m \cdot \Delta$$

$$\underbrace{\sum_e C_e}_{\text{capacity of the cut}} < F + m \cdot \Delta$$

( END OF PROOF OF CLAIM )

$F \leftarrow$  flow at the end of  $\Delta$  phase

$F' \leftarrow$  flow at the end of  $\frac{\Delta}{2}$  phase

$$\underbrace{F + m \cdot \Delta}_{\text{any feasible flow is at most the capacity of any cut}} > F' \geq F + L \cdot \frac{\Delta}{2} \Rightarrow \boxed{L < 2m}$$

## BIPARTITE MATCHING

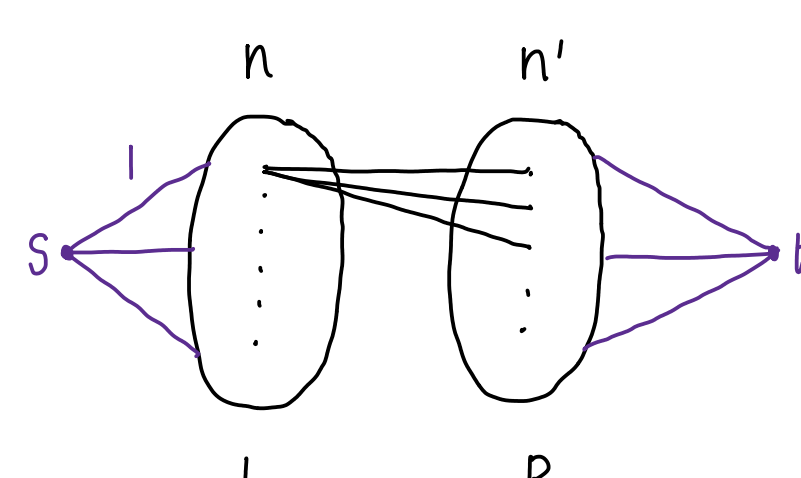
$M \subseteq E$

such that  $v \in$  only one edge in the set  $M$ .

Perfect matching = every vertex has an incident edge in  $M$ .

- Add  $s, t$  and add edges from :  $s$  to every vertex in  $L$   
every vertex in  $R$  to  $t$ .

- Assign capacity 1 to each edge.



Obs: Max. flow gives max. matching.